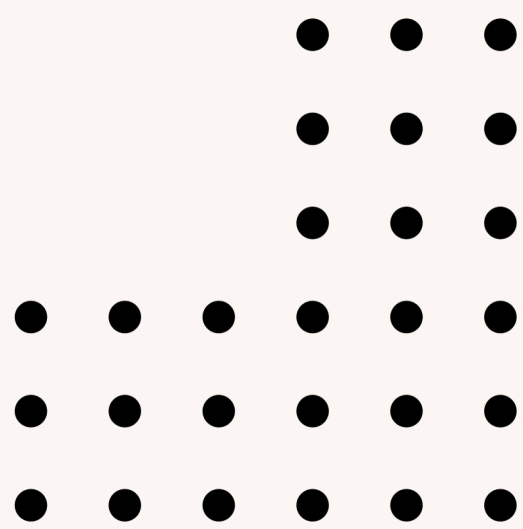
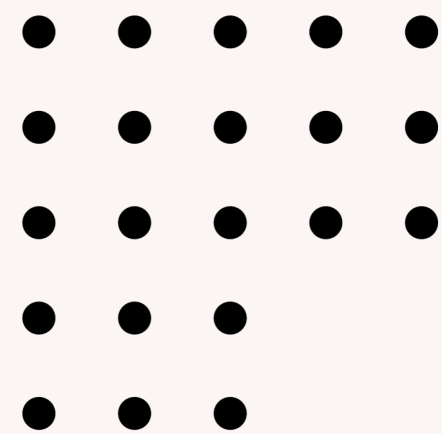




Projeto de LP1/UERJ atribuído pelo
Professor Francisco Sant'Anna

SISTEMA DE GERENCIAMENTO DE REGISTROS

Realizado por
Daniel Brum, Gabriel Schineider e Rafael Osborne



RESUMO DA APLICAÇÃO

Um modelo de dados
para a administração de
estoque de uma
concessionária.



INFORMAÇÕES GERENCIADAS

As informações gerenciadas pelo sistema baseiam-se no maquinário vendido pela concessionária, isto é:

- Motos
 - Pressão (2 pneus)
 - Tipo de guidom
 - Offroad
- Carros
 - Tração
 - Portas
 - Pressão(4 pneus)
 - Elétrico
- Comuns:
 - Preço, cor, modelo*, combustível*
- Barcos
 - Litragem
 - V Máxima
 - Casco
- Helicópteros
 - Passageiros
 - Altitude
 - Pás
- Bicicletas
 - Marchas
 - Suspensão
 - Diâmetro de roda
- Skates
 - Tipo de roda
 - Comprimento
 - Diâmetro de roda

OPERAÇÕES REALIZADAS

- 1 Criação de Registros Busca de Registros
- 2 Edição de Registros Remoção de Registros
- 3 Todas essas operações são realizadas dentro
- 4 de um arquivo que armazena os registros
- *

CRIAÇÃO DE REGISTROS

Consiste na escrita de um registro para o documento criado e gerenciado pela aplicação. Sendo assim possível armazenar o registro no documento.

```
void EscreverEntrada(ENTRADA_FINAL teste, FILE* f){
    //Escreve uma entrada na ID informada
    //Assume todas entradas de tamanho igual(144), FILE* em modo rb+ ou wb+
    //Esse metodo de calculo impede de tentar truncar strings com menos de 25 chars
    //Em troca, acesso de entradas fica em O(1) em vez de O(n)
    fseek(f,(long int) (teste.ID-1)*8,SEEK_SET); //atualizar tabela de IDs e indices
    fwrite(&teste.ID,sizeof(int),1,f);
    fwrite(&teste.indice,sizeof(int),1,f);

    fseek(f,(long int) INDICE_INICIO+sizeof(ENTRADA_FINAL)*teste.indice,SEEK_SET); //escrever arquivo na posicao correta
    fwrite(&teste.ID,sizeof(int),1,f);
    fwrite(&teste.indice,sizeof(int),1,f);
    fwrite(&teste.APAGADO,sizeof(int),1,f);
    fwrite(&teste.classe,sizeof(int),1,f);
    fwrite(teste.nome,sizeof(char),ESPACO,f);
    fwrite(&teste.cor,sizeof(char),ESPACO,f);
    fwrite(&teste.preco,sizeof(int),1,f);
    if (teste.classe){
        escreveManual(teste.manual,f);
    }
    else {
        escreveMotorizado(teste.motorizado,f);
    }
    fflush(f);
}
```


BUSCA DE REGISTROS

Consiste na procura de um registro, informando o ID dele, no documento gerenciado pela aplicação.

```
void BuscarRegistro(FILE* f)
{
    // Assume-se a já existência do registro, cuja posição no arquivo é
    ENTRADA_FINAL registro; // Variável nova que receberá a cópia das i
    int id; // ID desse registro já existente
    puts("Informe o ID do registro que deseja buscar: ");
    scanf("%d", &id);

    LeEntrada(&registro, id, f);
    ExibeEntrada(registro);
    fflush(f);
}
```

```
void LeEntrada(ENTRADA_FINAL* teste, int id, FILE* f)
{
    //Lê a região do arquivo referente ao ID informado pela estrutura
    //Assume todas entradas de tamanho igual(144), FILE* em modo rb+ ou wb+
    //Esse metodo de calculo impede de tentar truncar strings com menos de 28 chars
    //Em troca, acesso de entradas fica em O(1) em vez de O(n)

    int indice=pegaIndice(id,f);
    if (indice==NULO){
        teste->ID=id;
        teste->indice=indice;
        teste->APAGADO=sim;
    }
    else{
        fseek(f, (long int) INDICE_INICIO+sizeof(ENTRADA_FINAL)*indice, SEEK_SET); // Posiciona o cursor do arquivo, a partir da posição inicial,

        fread(&(teste->ID), sizeof(int), 1, f);
        fread(&(teste->indice), sizeof(int), 1, f);
        fread(&(teste->APAGADO), sizeof(int), 1, f);
        fread(&(teste->classe), sizeof(int), 1, f);
        fread(teste->nome, sizeof(char), ESPACO, f);
        fread(teste->cor, sizeof(char), ESPACO, f);
        fread(&(teste->preco), sizeof(int), 1, f);
        if (teste->classe)
        {
            leManual(&(teste->manual),f);
        }
        else
        {
            leMotorizado(&(teste->motorizado),f);
        }
    }
}
```

EDIÇÃO DE REGISTROS

Consiste na alteração de algum campo de algum registro. Faz-se necessário passar o ID do registro para que a aplicação possa buscá-lo no documento gerenciado por ela.

```
void EditarRegistro(FILE* arquivo){  
    int id_editar;  
    printf("Informe a ID da entrada que voce quer editar:\n");  
    scanf("%d",&id_editar);  
    ENTRADA_FINAL modificado;  
  
    LeEntrada(&modificado,id_editar,arquivo);  
    ModificarEntrada(&modificado);  
    EscreverEntrada(modificado,arquivo);  
    printf("Entrada modificada:\n");  
    ExibeEntrada(modificado);  
}
```

REMOÇÃO DE REGISTROS

Consiste em retirar do documento algum registro específico. Faz-se necessário passar o ID do registro para que a aplicação possa buscá-lo no documento gerenciado por ela para, então, removê-lo.

```
void RemoverRegistro(int indice, FILE *f) //só muda o campo APAGADO pra 1, não remove realmente do arquivo
{
    int removido = 1;

    long int PosicaoDoRegistro = INDICE_INICIO+sizeof(ENTRADA_FINAL) * indice;
    long int PosicaoParaRemover = (PosicaoDoRegistro + 2*sizeof(int)); //pula o ID

    fseek(f,PosicaoParaRemover, SEEK_SET);
    fwrite(&removido, sizeof(int), 1, f);

    fflush(f); //só pra garantir que o dado foi escrito no arquivo.
}
```

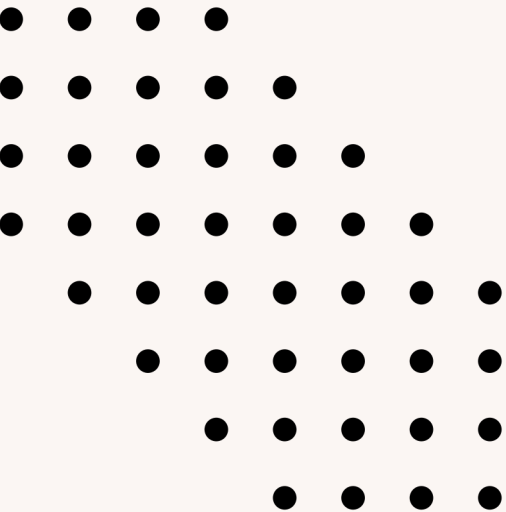

COMO ESSE SISTEMA AJUDA UMA CONCESSIONÁRIA?

Imagine uma loja que compra e vende diversos veículos todas semanas.

Com esse sistema, não há mais a necessidade de diversos papéis para cada veículo, algo que poderia ficar confuso e estressante em pouco tempo, devido alto cuidado necessário e à falta de flexibilidade característicos das folhas.

Apenas tendo as informações, faz-se possível administrar cada detalhe de cada registro. Ou seja, nosso sistema facilita a organização e torna tão simples quanto eficiente o processo de gerenciamento.





Caso nosso Sistema tenha interessado-lhe,
entre em contato!*

@nossoProjeto-De-LP1
www.projetoSGRC.com.br
(21) 91111-1111

